Szoftver fejlesztés és tesztelés

# D&D Ultimate Tool
# Projekt dokumentáció


**Készítette**
Adeniran Ádám Kristóf
László Tamás
Szoftverfejlesztő és -tesztelő szak

**Témavezető**
Kerényi Róbert Nándor
oktató


Miskolc, 2026

# Tartalomjegyzék


**1. Bevezetés** **4**

1.1. A projekt bemutatása és célja . . . . . 4

1.2. Célközönség meghatározása . . . . . 5

1.3. Alkalmazott technológiák és eszközök . . . . . 5

**2. Technológiák** **6**

2.1. Használt technológiák . . . . . 6

2.1.1. Backend . . . . . 6

2.1.2. Adatbázis . . . . . 7

2.1.3. Frontend . . . . . 8

2.1.4. Fejlesztői eszközök . . . . . 9

**3. Követelményanalízis** **10**

3.1. Funkcionális követelmények . . . . . 10

3.2. Nem funkcionális követelmények . . . . . 10

3.3. Használati esetek (Use Cases) . . . . . 10

**4. Rendszertervezés** **12**

4.1. Adatbázis terv . . . . . 12

4.1.1. ER-diagram és kapcsolatok . . . . . 12

4.1.2. Adatszótár és táblaleírások . . . . . 12

4.2. Rendszerarchitektúra és komponensek . . . . . 12

4.2.1. ORM (Object-Relational Mapping) . . . . . 12

4.2.2. Entity Framework . . . . . 12

4.3. REST API végpontok tervezése . . . . . 12

4.4. UI/UX Tervek (Wireframes) . . . . . 13

**5. Megvalósítás** **14**

5.1. Adatbázis réteg implementációja . . . . . 14

5.2. REST Backend fejlesztése . . . . . 14

5.2.1. Adatkezelés és Üzleti logika . . . . . 14

5.2.2. Biztonság és Autentikáció (JWT) . . . . . 14

5.3. React Frontend fejlesztése . . . . . 14
5.3.1. Komponens-architektúra . . . . . 14
5.3.2. Reszponzív megoldások és Mobil nézet . . . . . 14
5.4. WPF Desktop alkalmazás . . . . . 15

**6. Minőségbiztosítás és Tesztelés** **16**
6.1. Unit tesztek (Egységtesztelés) . . . . . 16
6.2. API tesztelés (Postman/Swagger) . . . . . 16
6.3. Felhasználói felület és Reszponzivitás tesztelése . . . . . 16
6.4. Hibajegyzék és javítások . . . . . 16

**7. Felhasználói útmutató** **17**
7.1. Rendszerkövetelmények és Telepítés . . . . . 17
7.2. A szoftver használata (Web/WPF) . . . . . 17

**8. Összegzés** **19**
8.1. Projekt értékelése . . . . . 19
8.2. Továbbfejlesztési lehetőségek . . . . . 19

**Mellékletek** **21**

**Irodalomjegyzék** **23**

# 1. fejezet

# Bevezetés

## 1.1. A projekt bemutatása és célja

A **D&D Ultimate Tool** egy webes alkalmazás, amely támogatja a **Dungeons & Dragons** játékosokat és mesélőket kampányok, karakterek és közösségi funkciók kezelésében. A rendszer célja egy egységes, többmodulos felület biztosítása a játék előkészítéséhez és lebonyolításához, valós idejű kommunikációval és bővíthető adatmodellel.

1.1. ábra. D&D Ultimate Tool logó

## 1.2. Célközönség meghatározása

A célcsoport az aktív D&D közösség:

- játékosok, akik karaktereiket és felszereléseiket strukturáltan szeretnék kezelni,
- mesélők (DM-ek), akik encounter-t, kampányt és jegyzeteket vezetnek,
- baráti társaságok, akik online szervezik a játékalkalmakat és kommunikációt.

## 1.3. Alkalmazott technológiák és eszközök

Mellékletre hivatkozás:
Lásd: 8.1 melléklet ábrát.

- **Adatbázis:** MySQL
- **Backend:** ASP.NET Core Web API, Entity Framework
- **Webes kliens:** React (reszponzív kialakítás)
- **Fejlesztői eszközök:** Git/GitHub, VS Code, Visual Studio, Postman/Swagger

# 2. fejezet

# Technológiák

## 2.1. Használt technológiák

A **D&D Ultimate Tool** fejlesztése során olyan technológiai stack-et alkalmaztunk, amely biztosítja a **skálázhatóságot**, a **biztonságos adatkezelést**, valamint a **gyors és modern felhasználói élményt**. A rendszer architektúrája három fő pillérre épül: **backend** (API és üzleti logika), **frontend** (felhasználói felület) és **adatbázis** (perzisztens adattárolás).

A megoldás alapelve, hogy a kliensoldali alkalmazás (React) kizárólag a megjelenítésért és a felhasználói interakciókért felel, míg az üzleti logika és az adat-hozzáférés a szerveren (ASP.NET Core Web API) történik. Ez a szétválasztás (*separation of concerns*) könnyebb karbantartást, jobb tesztelhetőséget és rugalmas bővíthetőséget eredményez.

Hiányzó kép:
`architecture`

2.1. ábra. A rendszer áttekintése: React frontend – ASP.NET Core API – MySQL adatbázis

### 2.1.1. Backend

A backend megvalósításához **ASP.NET Core Web API**-t használunk, amely magas teljesítményű és platformfüggetlen keretrendszer. A backend fő felelősségei:

- **Autentikáció és jogosultságkezelés:** JWT alapú beléptetés, védett végpontok.
- **Üzleti logika:** karakterek, kampányok, wiki bejegyzések és dobások kezelése.
- **Chat és közösségi funkciók:** üzenetküldés és kapcsolati logika (barát rendszer).

– **Adat-hozzáférés:** entitások kezelése és lekérdezések végrehajtása.

A réteges felépítés célja, hogy a kontroller réteg csak a HTTP kérések kezelésére szolgáljon, míg a tényleges üzleti szabályok külön szolgáltatási (service) rétegben helyezkedjenek el. Ez csökkenti a kód ismétlődését, és megkönnyíti a tesztelést.

#### ASP.NET Core

Az **ASP.NET Core** moduláris felépítése lehetővé teszi a middleware alapú kérésfeldolgozást, ami jól használható például naplózás, CORS, hibakezelés, illetve autentikáció beállítására. A projektben különösen fontos, hogy az API-k konzisztens formában szolgáltassanak adatot a frontend felé (jellemzően JSON formátumban), és stabilan kezeljék a hibás kéréseket.

Az alábbi példa egy tipikus végpont szerkezetét szemlélteti (szemléltető jelleggel):

```
[HttpGet("campaigns")]
[Authorize]
public IActionResult GetCampaigns() {
        // Lekérdezés és visszaadás DTO-ként
        return Ok(new { Items = new [] { "
           Campaign A", "Campaign B" } });
}
```

Listing 2.1. Példa API végpont (szemléltető)

#### Entity Framework Core

A perzisztencia rétegben az **Entity Framework Core** (EF Core) szerepe, hogy a C# objektumokat és az adatbázis táblákat összekösse, így az adatkezelés a domain modell szintjén történhet. Az EF Core támogatja a migrációkat is, ami lehetővé teszi, hogy a fejlesztés során az adatbázis-séma verziózottan, reprodukálható módon változzon.

#### Autentikáció (JWT)

A felhasználói munkamenet kezeléséhez **JWT** (JSON Web Token) alapú autentikációt alkalmazunk. A bejelentkezés során a szerver tokent ad vissza, amelyet a kliens a következő kérések során a `Authorization: Bearer <token>` fejlécben továbbít. Ez biztosítja, hogy a védett erőforrásokat csak az arra jogosult felhasználók érjék el.

### 2.1.2. Adatbázis

Az adatok tárolására **MySQL** relációs adatbázist alkalmazunk. A relációs modell előnye, hogy strukturált, konzisztens adattárolást biztosít, és jól támogatja a kapcsolatok

kezelését (felhasználók–barátok, kampányok–játékosok, üzenetek–felhasználók, stb.). A táblák közötti kapcsolatok idegen kulcsokkal és indexekkel optimalizálhatók, ami különösen fontos nagyobb adatmennyiség és gyakori lekérdezések esetén.

A projektben tipikus entitások:

- **Users**: felhasználói adatok
- **Friends**: barát kapcsolatok (állapotokkal)
- **Messages**: privát és csoportos üzenetek
- **Campaigns / Characters**: kampányok és karakterek alapadatai

A fejlesztés során az adatbázis kezeléséhez használható eszköz például **phpMyAdmin** vagy **MySQL Workbench**. A cél a jól átgondolt, normalizált struktúra, amely csökkenti az ismétlődést, és támogatja az adatintegritást.

### 2.1.3. Frontend

A felhasználói felületet **React** alapokon valósítjuk meg, amely komponens-alapú fejlesztést tesz lehetővé. A React előnye, hogy a UI logika újrafelhasználható elemekből épül fel, és a felület gyorsan frissíthető állapotváltozások hatására. A projektben a frontend tipikusan az alábbi feladatokat látja el:

- **Oldalak és nézetek:** kampányok, karakterlap, wiki, chat.
- **Állapotkezelés:** felhasználói session, kiválasztott kampány/karakter, UI állapotok.
- **API kommunikáció:** HTTP kérések kezelése, token továbbítása, hibák megjelenítése.
- **Reszponzív megjelenés:** mobil és asztali nézet támogatása.

#### React

A React ökoszisztémája támogatja a kliensoldali routingot (pl. React Router), valamint bővíthető állapotkezelési megoldásokat (Context API / Redux). A komponens-alapú szerkezet segíti, hogy az alkalmazás egyes részei (pl. chat panel, karakterkártyák, kampánylista) külön fejleszthetők és karbantarthatók legyenek.

```
import React from "react";

export default function CampaignCard({ name,
    members }) {
```

```
                    return (
                    <div className="card">
                    <h3>{name}</h3>
                    <p>Játékosok: {members}</p>
                    </div>
                    );
                }
```

Listing 2.2. Példa React komponens (szemléltető)

### 2.1.4. Fejlesztői eszközök

A fejlesztés során az alábbi eszközök támogatják a hatékony munkavégzést és a minőségbiztosítást:

- **Git & GitHub:** verziókezelés, branch stratégia, pull request alapú fejlesztés.
- **Visual Studio / VS Code:** backend és frontend fejlesztés, debug és refaktor eszközök.
- **Postman / Swagger:** API végpontok ellenőrzése és dokumentálása.
- **Adatbázis kliens:** phpMyAdmin / MySQL Workbench (opcionális).

# 3. fejezet

# Követelményanalízis

## 3.1. Funkcionális követelmények

- **Karakterkezelés:** karakterek létrehozása, módosítása, mentése
- **Kampánykezelés:** kampányok, kalandnapló, jegyzetek
- **Dobáskalkulátor:** testreszabható kockadobás
- **Wiki modul:** világ és homebrew tartalom dokumentálása
- **DM Tools:** kezdeményezés, NPC/szörny adatbázis, encounter támogatás
- **Barát rendszer:** kapcsolatok kezelése
- **Chat:** privát üzenetek és csoportos beszélgetések

## 3.2. Nem funkcionális követelmények

- Reszponzív felület (mobil és desktop)
- Biztonságos hitelesítés és jogosultságkezelés (JWT)
- Skálázható, réteges architektúra
- Karbantartható kódbázis (elkülönített frontend/backend)

## 3.3. Használati esetek (Use Cases)

Példák:

- Felhasználó regisztrál és bejelentkezik.

– Mesélő kampányt hoz létre, játékosokat meghív.

– Játékos karaktert készít, dobást indít, és chatben egyeztet.

# 4. fejezet

# Rendszertervezés

## 4.1. Adatbázis terv

### 4.1.1. ER-diagram és kapcsolatok

Az adatbázis relációs modellre épül. A barát rendszer és üzenetek kezeléséhez külön táblák tartoznak (barátkapcsolat, üzenetnapló).

### 4.1.2. Adatszótár és táblaleírások

A főbb entitások: Users, Friends, Messages, Campaigns, Characters, WikiEntries.

## 4.2. Rendszerarchitektúra és komponensek

### 4.2.1. ORM (Object-Relational Mapping)

Entity Framework használata a domain modellek és a MySQL relációk leképzéséhez.

### 4.2.2. Entity Framework

Code First / Database First megközelítés a projekt ütemezésétől függően.

## 4.3. REST API végpontok tervezése

Tipikus végpontok:

- `POST /api/auth/login`
- `GET /api/campaigns`
- `POST /api/messages`

## 4.4. UI/UX Tervek (Wireframes)

A fő képernyők: bejelentkezés, dashboard, kampány nézet, karakterlap, chat.

# 5. fejezet

# Megvalósítás

## 5.1. Adatbázis réteg implementációja

MySQL táblák, kulcsok és indexek kialakítása, adatkonzisztencia szabályokkal.

## 5.2. REST Backend fejlesztése

### 5.2.1. Adatkezelés és Üzleti logika

Szolgáltatási réteg (service layer) felel az üzleti szabályokért, controller réteg a végpontokért.

### 5.2.2. Biztonság és Autentikáció (JWT)

JWT alapú tokenkezelés, jogosultsági szintek és védett végpontok.

## 5.3. React Frontend fejlesztése

### 5.3.1. Komponens-architektúra

Komponens alapú felépítés, külön modulok a karakter, kampány és chat területekhez.

### 5.3.2. Reszponzív megoldások és Mobil nézet

Grid/flex alapú elrendezés, mobilbarát menü és töréspontok.

```
import React from 'react';

function Welcome(props) {
        return <h1>Hello, {props.name}</h1>;
}
```

```
6
7                    export default Welcome;
```

Listing 5.1. Példa React komponens

## 5.4. WPF Desktop alkalmazás

(Amennyiben tartozik a projekthez.) MVVM struktúra, adatkezelés és kiegészítő admin modulok.

# 6. fejezet

# Minőségbiztosítás és Tesztelés

## 6.1. Unit tesztek (Egységtesztelés)

Backend: xUnit vagy NUnit; kritikus üzleti logikák tesztelése.

## 6.2. API tesztelés (Postman/Swagger)

Swagger UI dokumentáció; Postman gyűjtemények regresszióhoz.

## 6.3. Felhasználói felület és Reszponzivitás tesztelése

Kézi és automatizált UI ellenőrzések (pl. Lighthouse).

## 6.4. Hibajegyzék és javítások

Verziózott hibakezelés GitHub Issues-ban.

# 7. fejezet

# Felhasználói útmutató

## 7.1. Rendszerkövetelmények és Telepítés

- Modern böngésző (Chrome/Firefox/Edge)
- Backend futtatás: .NET SDK és MySQL

## 7.2. A szoftver használata (Web/WPF)

1. Böngészőben megnyitás
2. Bejelentkezés
3. Kampány/karakter létrehozás
4. Chat és eszközök használata játék közben

7.1. ábra. Bejelentkezési képernyő

# 8. fejezet

# Összegzés

## 8.1. Projekt értékelése

A D&D Ultimate Tool célja egy olyan komplex rendszer biztosítása, amely egységes felületen támogatja a kampánykezelést, karakteradminisztrációt, dobásokat és közösségi funkciókat.

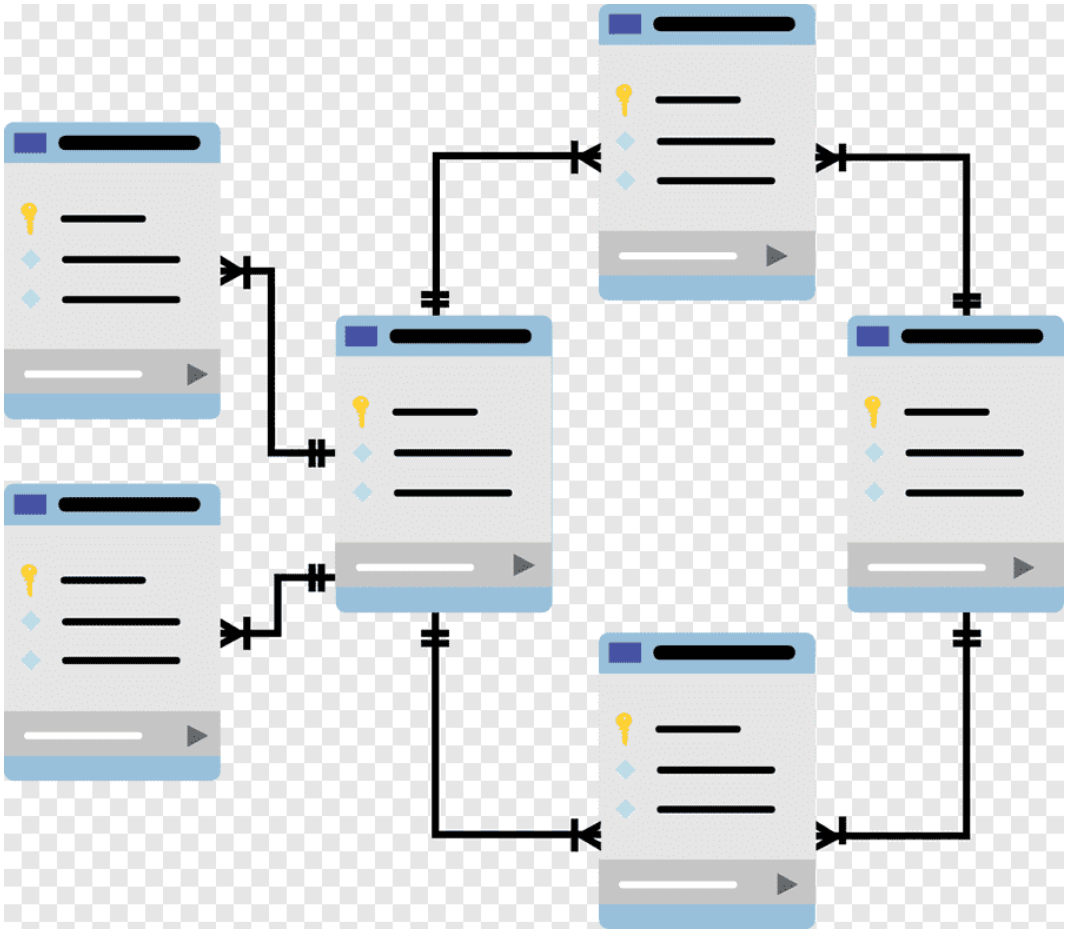
## 8.2. Továbbfejlesztési lehetőségek

- Mobilbarát UI további fejlesztése
- Discord bot integráció
- Szabálykönyv-import és verziókövetés
- Többnyelvű felület támogatása

# Köszönetnyilvánítás

Köszönet mindazoknak, akik hozzájárultak a projekt elkészítéséhez, külön megköszönve ...

# Mellékletek



8.1. ábra. Adatbázis ábra

EZ EGY MELLÉKLET!!!

8.2. ábra. Szkennelt dokumentum

# Irodalomjegyzék

[1] Fazekas István: *Valószínűségszámítás*, Debreceni Egyetem, Debrecen, 2004.

[2] Tómács Tibor: *A valószínűségszámítás alapjai*, Líceum Kiadó, Eger, 2005.

[3] Várterész Magdolna: *Az informatika logika alapjai előadások*, 2006/07-es tanév 1.félév, http://users.atw.hu/de-mi/index.php?r=file/download&id=219, Letöltve: 2019 március 12.-én